



UNS
UNIVERSIDAD
NACIONAL DEL SANTA

CURSO DE APLICACIONES MÓVILES

Desarrolla. Conecta. Innova.
Lleva tus ideas al siguiente nivel.



Flutter



Kotlin



React Native



Android



iOS

```
import 'package:flutter  
class HomePage extends  
@override  
Widget build(BuildContext  
return Scaffold(  
  appBar: AppBar(  
    body: Center(  
      child: Text(  
        'Hola, Mundo!!  
    )  
  )  
)  
)
```



**DESARROLLO
MULTIPLATAFORMA**
Crea apps para Android
e iOS con un solo código.



**INTERFACES
MODERNAS**
Diseña experiencias
atractivas y funcionales.



**CONECTIVIDAD
Y SERVICIOS**
Integra APIs, bases de datos
y servicios en la nube.



**SEGURIDAD
Y RENDIMIENTO**
Desarrolla aplicaciones
seguras, rápidas y
optimizadas.



**PUBLICACIÓN
Y DESPLIEGUE**
Publica tu app en Google
Play y App Store.



DESARROLLO
DE APPS



UI/UX
MÓVIL



APIS Y
SERVICIOS



BASES DE
DATOS



PRUEBAS Y
DEPLOY



Docente:

Dr. Mirko Manrique Ronceros

INTRODUCCIÓN

En el desarrollo de aplicaciones móviles, la interacción entre el usuario y la aplicación es un elemento esencial para garantizar una experiencia dinámica y funcional. Para lograrlo, las aplicaciones deben responder a diferentes acciones realizadas por el usuario, como pulsar botones, seleccionar opciones o desplazarse entre pantallas. Estas interacciones se gestionan mediante eventos, los cuales permiten detectar y ejecutar acciones específicas dentro de la aplicación.

Asimismo, las aplicaciones móviles modernas requieren mecanismos que faciliten la comunicación entre componentes y el intercambio de información. En este contexto, los Intents constituyen una herramienta fundamental, ya que permiten iniciar actividades, compartir datos y establecer comunicación entre diferentes pantallas o aplicaciones dentro del sistema operativo móvil.

Por otro lado, la navegación básica es indispensable para organizar el flujo de interacción del usuario dentro de la aplicación. Una correcta navegación permite desplazarse de manera clara y ordenada entre las distintas interfaces, mejorando la usabilidad y la experiencia general del usuario. El estudio de eventos, intents y navegación básica proporciona las bases necesarias para desarrollar aplicaciones móviles interactivas, estructuradas y fáciles de utilizar..

En Android, una aplicación móvil no solo muestra pantallas; también responde a acciones del usuario, se comunica entre componentes y permite desplazarse entre diferentes interfaces. Para lograrlo, Android utiliza tres conceptos fundamentales:

- **Eventos:** acciones generadas por el usuario o el sistema.
- **Intents:** objetos de mensajería para solicitar acciones entre componentes.
- **Navegación:** desplazamiento entre pantallas, Activities o Fragments.

Estos temas son esenciales para construir aplicaciones móviles interactivas, funcionales y correctamente estructuradas.

I. EVENTOS EN ANDROID

1.1. ¿Qué es un evento?

Un evento es una acción que ocurre durante la ejecución de una aplicación. Puede ser originado por:

- El usuario.
- El sistema operativo.
- Un componente de la aplicación.

Ejemplos comunes:

- Presionar un botón.
- Escribir en un campo de texto.
- Seleccionar un elemento de una lista.
- Tocar una imagen.
- Cambiar la orientación del dispositivo.
- Recibir una notificación.
- Perder conexión a internet.

En Android, los eventos permiten que la aplicación reaccione de forma dinámica ante la interacción del usuario.

Referencia oficial: Android explica que las apps pueden responder a eventos de entrada como gestos, teclado, botones físicos y pantalla táctil mediante APIs del framework.

1.2. Tipos de eventos

a) Eventos de clic

Son los más comunes. Se producen cuando el usuario presiona un botón, imagen, tarjeta u otro componente.

Ejemplo en Kotlin:

```
val btnGuardar = findViewById<Button>(R.id.btnGuardar)
btnGuardar.setOnClickListener {
    Toast.makeText(this, "Datos guardados",
        Toast.LENGTH_SHORT).show()
}
```

Ejemplo en Java:

```
Button btnGuardar = findViewById(R.id.btnGuardar);
btnGuardar.setOnClickListener(v -> {
    Toast.makeText(this, "Datos guardados",
        Toast.LENGTH_SHORT).show();
});
```

b) Eventos de texto

Se ejecutan cuando el usuario escribe, modifica o elimina contenido en un campo de texto.

Ejemplo en Kotlin:

```
val txtNombre = findViewById<EditText>(R.id.txtNombre)
txtNombre.addTextChangedListener {
    val nombre = it.toString()
}
```

Uso común:

- Validar formularios.
- Activar o desactivar botones.
- Filtrar listas.
- Buscar información en tiempo real.

c) Eventos de selección

Ocurren cuando el usuario selecciona un elemento de un listado, menú o spinner.

Ejemplo:

```
spinner.onItemSelectedListener = object :
    AdapterView.OnItemSelectedListener {
        override fun onItemSelected(
            parent: AdapterView<*>?,
            view: View?,
            position: Int,
            id: Long
        ) {
            val opcion = parent?.getItemAtPosition(position).toString()
        }
        override fun onNothingSelected(parent: AdapterView<*>?) {}
    }
```

d) Eventos táctiles

Permiten capturar gestos más específicos:

- Tocar.
- Mantener presionado.
- Deslizar.
- Arrastrar.
- Hacer zoom.

Ejemplo:

```
view.setOnTouchListener { _, event ->
    when (event.action) {
        MotionEvent.ACTION_DOWN -> {
            true
        }
        MotionEvent.ACTION_UP -> {
            true
        }
        else -> false
    }
}
```

e) Eventos del ciclo de vida

Una Activity o Fragment también genera eventos internos cuando cambia de estado.

Ejemplo en una Activity:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
}
```

```
override fun onStart() {  
    super.onStart()  
}
```

```
override fun onResume() {  
    super.onResume()  
}
```

```
override fun onPause() {  
    super.onPause()  
}
```

Estos eventos permiten controlar qué sucede cuando:

- Se abre una pantalla.
- La app pasa a segundo plano.
- El usuario regresa a la app.
- La pantalla se destruye.

1.3. Listener

Un listener es un mecanismo que “escucha” un evento y ejecuta una acción cuando dicho evento ocurre.

Ejemplo conceptual:

Elemento	Función
Botón	Componente visual
Clic	Evento
Listener	Escucha el evento
Código interno	Acción ejecutada

Ejemplo:

```
btnLogin.setOnClickListener {  
    iniciarSesion()  
}
```

Aquí, `setOnClickListener` espera el clic del usuario y luego ejecuta la función `iniciarSesion()`.

1.4. Importancia de los eventos

Los eventos permiten:

- Crear aplicaciones interactivas.
- Validar datos ingresados por el usuario.
- Ejecutar acciones según la interacción.
- Mejorar la experiencia de usuario.
- Controlar el comportamiento de la interfaz.

Sin eventos, una aplicación sería estática y no respondería al usuario.

II. INTENTS EN ANDROID**2.1. ¿Qué es un Intent?**

Un Intent es un objeto de mensajería que permite solicitar una acción a otro componente de Android. Puede utilizarse para iniciar una Activity, enviar información, iniciar servicios o solicitar acciones a otras aplicaciones.

En términos simples, un Intent funciona como un “mensaje” que indica:

- Qué acción se desea realizar.
- Qué componente debe ejecutarse.
- Qué datos se enviarán.

2.2. Usos principales de los Intents

Según la documentación oficial de Android, los Intents se utilizan principalmente para:

- Iniciar una Activity.
- Iniciar un Service.
- Entregar un Broadcast.

Ejemplos:

Uso	Ejemplo
Abrir otra pantalla	LoginActivity → HomeActivity
Enviar datos	Enviar nombre de usuario
Abrir otra app	Abrir navegador, cámara o mapas
Compartir contenido	Enviar texto a WhatsApp o correo
Ejecutar servicio	Iniciar tarea en segundo plano

2.3. Tipos de Intents

a) Intent explícito

Se utiliza cuando se conoce exactamente el componente que se desea abrir.

Ejemplo: pasar de MainActivity a DetalleActivity.

```
val intent = Intent(this, DetalleActivity::class.java)
startActivity(intent)
```

En Java:

```
Intent intent = new Intent(this, DetalleActivity.class);
startActivity(intent);
```

Este tipo de Intent se usa comúnmente dentro de la misma aplicación.

b) Intent implícito

Se utiliza cuando no se especifica directamente el componente, sino la acción que se desea realizar.

Ejemplo: abrir una página web.

```
val intent = Intent(Intent.ACTION_VIEW)
intent.data = Uri.parse("https://developer.android.com")
startActivity(intent)
```

Android busca qué aplicación puede ejecutar esa acción, como Chrome, Edge, Firefox u otro navegador.

Referencia oficial: Android señala que los Intents implícitos declaran una acción general y permiten que otra app compatible la gestione.

2.4. Envío de datos entre Activities

Los Intents permiten enviar datos mediante putExtra.

Ejemplo:

```
val intent = Intent(this, BienvenidaActivity::class.java)
intent.putExtra("nombre", "Juan")
intent.putExtra("edad", 25)
startActivity(intent)
```

Recepción de datos:

```
val nombre = intent.getStringExtra("nombre")
val edad = intent.getIntExtra("edad", 0)
```

2.5. Ejemplo completo: Login básico

MainActivity

```
class MainActivity : AppCompatActivity() {
```

```
    private lateinit var txtUsuario: EditText
```

```
    private lateinit var btnIngresar: Button
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_main)
```

```
        txtUsuario = findViewById(R.id.txtUsuario)
```

```
        btnIngresar = findViewById(R.id.btnIngresar)
```

```
        btnIngresar.setOnClickListener {
```

```
            val usuario = txtUsuario.text.toString()
```

```
            if (usuario.isNotEmpty()) {
```

```
                val intent = Intent(this, HomeActivity::class.java)
```

```
                intent.putExtra("usuario", usuario)
```

```
                startActivity(intent)
```

```
            } else {
```

```
                Toast.makeText(this, "Ingrese usuario", Toast.LENGTH_SHORT).show()
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

HomeActivity

```

class HomeActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_home)

        val usuario = intent.getStringExtra("usuario")

        val txtBienvenida = findViewById<TextView>(R.id.txtBienvenida)

        txtBienvenida.text = "Bienvenido, $usuario"

    }

}

```

2.6. Ejemplos de Intents implícitos***Abrir navegador***

```

val intent = Intent(Intent.ACTION_VIEW,
    Uri.parse("https://www.google.com"))
startActivity(intent)

```

Realizar llamada

```

val intent = Intent(Intent.ACTION_DIAL)
intent.data = Uri.parse("tel:987654321")
startActivity(intent)

```

Enviar correo

```

val intent = Intent(Intent.ACTION_SENDTO).apply {
    data = Uri.parse("mailto:correo@ejemplo.com")
    putExtra(Intent.EXTRA_SUBJECT, "Consulta")
    putExtra(Intent.EXTRA_TEXT, "Estimado docente...")
}
startActivity(intent)

```

```

    }

    startActivity(intent)

Compartir texto

    val intent = Intent(Intent.ACTION_SEND).apply {
        type = "text/plain"
        putExtra(Intent.EXTRA_TEXT, "Contenido para compartir")
    }

    startActivity(Intent.createChooser(intent, "Compartir con"))

```

2.7. Intent Filters

Un Intent Filter permite declarar qué tipos de Intents puede recibir una Activity, Service o BroadcastReceiver.

Ejemplo en AndroidManifest.xml:

```

<activity android:name=".WebActivity">
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />

        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="android.intent.category.BROWSABLE" />

        <data android:scheme="https" />
    </intent-filter>
</activity>

```

Los filtros de Intent permiten que Android determine qué componente puede responder a una acción determinada.

2.8. Buenas prácticas con Intents

- Usar Intents explícitos para navegar dentro de la misma aplicación.
- Usar Intents implícitos para solicitar acciones a otras apps.
- Validar que exista una aplicación capaz de resolver el Intent.
- No enviar información sensible sin protección.
- Usar constantes para las claves de putExtra.
- Evitar enviar objetos pesados entre pantallas.

Ejemplo de validación:

```
if (intent.resolveActivity(packageManager) != null) {  
    startActivity(intent)  
}
```

III. NAVEGACIÓN BÁSICA EN ANDROID

3.1. Qué es la navegación?

La navegación es el proceso mediante el cual el usuario se desplaza entre pantallas o secciones de una aplicación.

Ejemplos:

Login → Home.

Home → Detalle.

Lista de eventos → Detalle del evento.

Perfil → Configuración.

Android considera la navegación como una parte central de la experiencia del usuario y recomienda mantener patrones consistentes e intuitivos.

3.2. Formas básicas de navegación

a) Navegación entre Activities

Es la forma más simple.

```
val intent = Intent(this, SegundaActivity::class.java)
startActivity(intent)
```

Para cerrar la Activity actual:

```
finish()
```

Ejemplo:

```
btnSalir.setOnClickListener {
    finish()
}
```

b) Navegación entre Fragments

Los Fragments permiten dividir la interfaz en secciones reutilizables dentro de una misma Activity.

Ejemplo conceptual:

MainActivity

- |— HomeFragment
- |— ScheduleFragment
- |— SpeakersFragment
- |— UbicationFragment

La navegación entre Fragments es más recomendable cuando una aplicación tiene varias secciones internas dentro de una pantalla principal.

c) Navigation Component

Navigation Component es una biblioteca de Android Jetpack que permite gestionar la navegación de manera estructurada. Proporciona beneficios como navegación entre destinos, animaciones, manejo del botón atrás y paso de argumentos.

Componentes principales:

Uso	Ejemplo
Abrir otra pantalla	LoginActivity → HomeActivity
Enviar datos	Enviar nombre de usuario
Abrir otra app	Abrir navegador, cámara o mapas
Compartir contenido	Enviar texto a WhatsApp o correo
Ejecutar servicio	Iniciar tarea en segundo plano

3.3. Navigation Graph

El Navigation Graph define visualmente las pantallas y rutas de navegación.

Ejemplo:

```
<navigation
    android:id="@+id/nav_graph"
    app:startDestination="@id/homeFragment">
    <fragment
        android:id="@+id/homeFragment"
        android:name="com.example.app.HomeFragment"
        android:label="Inicio">
        <action
            android:id="@+id/action_home_to_detail"
            app:destination="@id/detailFragment" />
    </fragment>
    <fragment
        android:id="@+id/detailFragment"
        android:name="com.example.app.DetailFragment"
        android:label="Detalle" />
</navigation>
```


3.4.NavHostFragment

El NavHostFragment es el contenedor donde se cargan los Fragments.

Ejemplo en XML:

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/nav_host_fragment"
    android:name="androidx.navigation.fragment.NavHostFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:navGraph="@navigation/nav_graph"
    app:defaultNavHost="true" />
```

3.5.NavController

El NavController ejecuta la navegación.

Ejemplo:

```
findNavController().navigate(R.id.action_home_to_detail)
```

Con argumentos:

```
val bundle = Bundle()
bundle.putString("titulo", "Evento Android")
findNavController().navigate(R.id.detailFragment, bundle)
```

Recepción:

```
val titulo = arguments?.getString("titulo")
```

3.6.Navegación con Bottom Navigation

Una aplicación moderna suele usar navegación inferior para secciones principales.

Ejemplo:

EVENTIFY

└─ Inicio

└─ Cronograma

└─ Ponentes

└─ Ubicación

XML:

```
<com.google.android.material.bottomnavigation.BottomNavigationView  
    android:id="@+id/bottomNavigation"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    app:menu="@menu/bottom_menu" />
```

Menú:

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">  
    <item  
        android:id="@+id/homeFragment"  
        android:title="Inicio"  
        android:icon="@drawable/ic_home" />  
    <item  
        android:id="@+id/scheduleFragment"  
        android:title="Cronograma"  
        android:icon="@drawable/ic_schedule" />  
    <item  
        android:id="@+id/speakersFragment"  
        android:title="Ponentes"  
        android:icon="@drawable/ic_person" />  
</menu>
```

CONFIGURACIÓN:

```
val navController = findNavController(R.id.nav_host_fragment)
val bottomNavigation = findViewById<BottomNavigationView>(R.id.bottomNavigation)
bottomNavigation.setupWithNavController(navController)
```

3.7. Back Stack

El Back Stack es la pila de pantallas visitadas por el usuario.

Ejemplo:

LoginActivity → HomeActivity → DetailActivity

Si el usuario presiona “Atrás”:

DetailActivity → HomeActivity

El componente Navigation gestiona el Back Stack para mantener una experiencia coherente. En Navigation 3, Android modela el desplazamiento usando una pila de referencias o claves de contenido.

IV. RELACIÓN ENTRE EVENTOS, INTENTS Y NAVEGACIÓN

Estos tres conceptos trabajan juntos.

Ejemplo:

Usuario presiona botón



Se genera un evento de clic



El listener ejecuta una acción



Se crea un Intent o se llama al NavController



La app navega a otra pantalla

Ejemplo práctico:

```
btnVerDetalle.setOnClickListener {
    val intent = Intent(this, DetalleActivity::class.java)
    intent.putExtra("idEvento", 10)
    startActivity(intent)
}
```

Interpretación:

Elemento	Representación
Clic en botón	Evento
setOnClickListener	Listener
Intent	Mensaje de navegación
startActivity	Cambio de pantalla
idEvento	Dato enviado

V. ERRORES COMUNES

Error	Explicación	Solución
Activity no declarada	No está en <code>AndroidManifest.xml</code>	Declararla correctamente
NullPointerException	Vista no inicializada	Revisar <code>findViewById</code>
Intent sin app receptora	No existe app compatible	Usar <code>resolveActivity</code>
Fragment no navega	Acción no definida	Revisar <code>nav_graph.xml</code>
Datos no llegan	Clave incorrecta	Usar constantes
App se cierra al volver	Mal manejo del Back Stack	Revisar navegación

VI. BUENAS PRÁCTICAS

- Usar Kotlin para reducir código repetitivo.
- Usar Navigation Component para apps con varios Fragments.
- Usar Intents explícitos para navegación interna.
- Usar Intents implícitos para interactuar con otras aplicaciones.

- Validar los datos antes de navegar.
- Evitar pasar objetos grandes entre Activities.
- Usar ViewModel para compartir datos entre Fragments.
- Mantener nombres claros para Activities, Fragments y acciones.
- Evitar lógica compleja dentro del OnClickListener.

CONCLUSIONES

Los eventos permiten que la aplicación responda a la interacción del usuario. Los Intents permiten comunicar componentes y solicitar acciones dentro o fuera de la aplicación. La navegación organiza el desplazamiento entre pantallas y mejora la experiencia del usuario.

En el desarrollo Android moderno, estos tres elementos son esenciales para construir aplicaciones funcionales, interactivas y escalables.